

MC521 - Relatório IX

Júlio da Silva Telles

22 de Junho de 2026

1 Course Schedule (CSES - 1679)

O problema exige a elaboração de um cronograma válido para concluir N cursos. São dadas M restrições na forma de pré-requisitos, indicando que um curso A deve ser obrigatoriamente concluído antes de um curso B . Caso seja impossível completar todos os cursos devido a dependências circulares, o programa deve acusar a impossibilidade.

1.1 Solução

O problema pode ser modelado como a busca por uma Ordenação Topológica (*Topological Sort*) em um Grafo Direcionado. A solução implementada alcança uma complexidade de tempo de $\mathcal{O}(V + E)$ e opera através dos seguintes passos:

1. **Modelagem do Grafo Invertido:** Durante a leitura da entrada, construímos o grafo invertendo a direção das arestas (`graph[v].PB(u)` em vez do tradicional $u \rightarrow v$). Essa estratégia dispensa a necessidade de inverter o vetor resposta ao final, já que a busca em profundidade preencherá os nós na ordem correta nativamente.
2. **Busca em Profundidade (DFS):** Iteramos sobre todos os nós do grafo. Se um nó ainda não foi visitado, iniciamos uma rotina recursiva de `dfs`. Cada curso finalizado pela recursão (quando todas as suas dependências foram resolvidas) é adicionado ao vetor `path`.
3. **Detecção de Ciclos:** Utilizamos um conjunto (`std::set<int> s`) para simular a pilha de recursão e rastrear os ancestrais do caminho atual. Ao visitar um vizinho, se ele já estiver contido em `s`, descobrimos uma "aresta de retorno" (*back-edge*), comprovando a existência de um ciclo. Nesse cenário, a flag `found` é definida como falsa e a resposta passa a ser `IMPOSSIBLE`.

2 Game Routes (CSES - 1681)

O cenário descreve um jogo com N níveis e M teletransportes unidirecionais entre eles. O objetivo é calcular o número total de rotas distintas possíveis começando do nível 1 e terminando no nível N . Por conta da quantidade potencialmente colossal de rotas, a resposta final deve ser expressa em módulo $10^9 + 7$.

2.1 Solução

Podemos resolver este problema aplicando Programação Dinâmica sobre um Grafo Direcionado Acíclico (DP em DAG), utilizando uma abordagem *Top-Down* (*Memoization*). A complexidade resultante é linear em relação ao tamanho do grafo, $\mathcal{O}(V + E)$. A execução transcorre da seguinte forma:

1. **Estado e Recursão:** A função recursiva `dfs` objetiva retornar quantas maneiras existem de ir do nó `u` atual até o nó de destino `n`. Se atingirmos o alvo (`u == n`), a base da recursão atua e retornamos o valor 1, que representa o término com sucesso de uma rota.
2. **Memoização:** Para evitar recalcular caminhos repetidos, o vetor de estados `p` guarda o número de rotas válidas computadas a partir daquele nó. Da mesma forma, o vetor `v` funciona como uma máscara de visitados tradicionais da DFS. Se um vizinho já foi processado no passado (`p[w] > 0`), seu valor acumulado é simplesmente somado em tempo constate.